Introduction to Software Engineering

Project Proposal & Plan



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives **1**
1 Member Contribution Assessment **2**
2 Vision **3**
2.1 Preliminary Problem Statement 3
2.2 Product Position Statement 4
2.3 Stakeholders and Users 4
2.4 Scope and Key Features 4
3 Proposed Solution **6**
3.1 Preliminary Architecture 6
3.2 Technology Stack 6
3.3 Hardware and Operating Environment 7
4 Project Plan **8**
4.1 Process and Phase Gates 8
4.2 Work Breakdown Structure (WBS) 9
4.3 Estimation 10
4.4 Schedule 14
4.5 Roles and Responsibilities 15
4.6 Human Resources and Cost Estimate 16
5 Risk Management **16**
6 Tools & Collaboration Setup **18**
7 AI Usage Log **20**

Project Proposal & Plan

Objectives

This document focus on the following topics:

- ✔ Completing the Project Proposal & Plan document with the following sections:
  - Vision
  - Proposed Solution
  - Project Plan
  - Risk Management
  - Tools & Collaboration Setup
- ✔ Including an AI usage log for this phase
- ✔ Understanding the Project Proposal & Plan document.

1 Member Contribution Assessment

ID	Name	Role(s)	Contribution (%)	Evidence
24127455	Lại Phan Nhật Nam	Team Lead	100	Jira_1 Jira_2 Commit
24127295	Phạm Hoàng Khôi Nguyên	Business Analyst	100	Jira
24127459	Trần Văn Nam	Designer	100	Jira_1 Jira_2 Commit
24127308	Nguyễn Hoàng Tiến	Developer	100	Jira Commit
24127409	Ngô Phúc Khang	Tester	100	Jira

2 Vision

2.1 Preliminary Problem Statement

Current situation and problem

Groups of friends, families, and small group funds often share costs for meals, groceries, or activities. Usually, members track these expenses through chat messages or spreadsheets. As more people take turns paying, it becomes harder to check which expenses were recorded, how much each person owes, and whether a repayment has actually been received. Manual updates can lead to missing records, inconsistent totals, and misunderstandings between members.

Proposed approach

SplitNest is a web application that keeps shared expenses and repayments in one place. After creating an account, users can create or join a private group, record expenses, select participating members, and see the amounts they owe or are owed.

For example, if one member pays 150,000 VND for a meal shared by three participants, each participant has a 50,000 VND share; only participants other than the expense payer owe a repayment to that payer.

Each amount owed is linked to the expense that created it, while the application can summarize the outstanding amounts for a group. A debtor reports that the full amount of a specific debt has been repaid outside SplitNest. The receiver then confirms or rejects the report. A pending repayment remains outstanding until confirmation. This avoids treating a self-reported transfer as verified while keeping the payment flow simple for the first release.

Expected Value

The goal is to provide a common and understandable record of group expenses, outstanding amounts, and settlement status. This should reduce the need to search through old messages or manually reconcile separate records. A group organizer or treasurer can follow contributions using the same shared information; SplitNest does not maintain a real bank fund or hold users' money.

Operating environment and implementation constraints

SplitNest will run in modern HTML5-compatible browsers on desktop and mobile devices with an internet connection. The proposed stack is React and JavaScript for the interface, Flask and Python for the backend, and MongoDB Atlas for data storage. Deployment is planned on Vercel (frontend) and Render (backend), using free-tier services where possible. The first version will work with whole-VND amounts, restrict access to members of each group, and store bank details only as text without connecting to banking APIs. With a five-person team and implementation scheduled for weeks 8–10, the project will prioritize the four core features defined below.

2.2 Product Position Statement

Element	Content
For	Friend groups, families and group-fund treasurers in Vietnam
Who	Regularly share expenses and need to see who owes whom and which repayments are confirmed.
The (product name)	SplitNest
That	Records group expenses in whole VND, tracks outstanding amounts, and supports a payer-reports / receiver-confirms settlement flow.
Unlike	General-purpose expense-sharing applications such as Splitwise, which serve broader use cases.
Our product	Focuses on a simple Vietnamese-first workflow and clear repayment confirmation; VietQR generation is an optional enhancement.

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Group organizer / treasurer	Creates a group, invites members, and may also participate in expenses.	Simple group setup and an overview of unpaid contributions.
2	Group member	Joins groups; records or shares expenses; may owe or receive repayments.	Accurate balances, accessible records, and clear payment statuses.
3	Debtor / receiver (situational roles)	The debtor reports a repayment; the member receiving the money confirms or rejects it. One user may play either role in different transactions.	Avoiding mistaken or repeated settlement updates.
4	Project team	Five students responsible for analysis, design, implementation, testing, and delivery.	A feasible scope and requirements that can be implemented and verified.

5	Instructor / TA	Academic stakeholders who review the project and evaluate its deliverables.	A clear problem, traceable scope, and a demonstrable working system.

2.4 Scope and Key Features

ID	Feature	Description	Priority (M/S/C/W)
F01	Accounts and bank details	Users can register, sign in, and optionally save bank name, account number, and account holder name as text. No bank account is connected.	Must
F02	Groups and invitations	Users can create private groups, invite registered members via a join link or QR code containing that link, and view group membership.	Must
F03	Shared expense recording	A group member can record an expense, its amount, one payer, and selected participants. The first release proposes equal splitting in whole VND with a consistent remainder rule.	Must
F04	Debt tracking and settlement	Group members can view amounts owed/receivable. A debtor reports repayment, and the receiving member confirms or rejects it before the record is settled.	Must
F05	VietQR payment code	Generate a VietQR code using saved bank details and a payment amount; transfers remain outside the application.	Should
F06	History and summary	Show recorded expenses and repayment activity in chronological order.	Could
F07	Debt reminders	Automated reminders for unpaid debts are deferred.	Won't
F08	Chats and notes	In-app messaging and debt-related notes are deferred.	Won't

The following decisions define the intended behavior of F02–F04 and will be expanded into detailed requirements, use cases in below:

Area	Refined rule for the initial release
Membership and access	Users must sign in before joining a group. Only members of a group can view its information or add expenses. When creating an expense, the person who paid and everyone sharing the cost must belong to that group. The QR code for joining a group simply opens an invitation link, so SplitNest does not need its own QR scanner.
Expense split	Each expense has one payer and at least one participant. The amount must be a positive whole number in VND. SplitNest divides the total amount equally among the selected participants. If the amount cannot be divided evenly, the system first gives each participant the same whole-VND amount. Any remaining dong are then distributed one by one in a fixed order, such as by member ID. For example, when splitting 100,000 VND among three people, one person pays 33,334 VND while the other two pay 33,333 VND each. This ensures that everyone pays nearly the same amount and the sum of all shares is exactly equal to the original expense.
Debt records	For each expense, SplitNest records how much each participant owes the person who paid. If the payer is also one of the participants, they do not owe money to themselves. The system uses these individual debts to calculate how much each member owes or is owed within the group. For the first version, debts between members are not automatically combined or canceled out.
Repayment status	Each repayment is made for one debt and must cover its full remaining amount. After the debtor marks it as paid, the system waits for the receiver to confirm. If confirmed, the debt is marked as settled. If rejected, it returns to unpaid status. Only the person receiving the money can confirm or reject a repayment, and the same debt cannot be settled twice.
Expense correction	Only the member who created an expense can edit or delete it. However, this is allowed only if no repayment request has been made for any debt related to that expense. Once a repayment request exists, the expense can no longer be changed or deleted, even if the request is later rejected. This helps prevent incorrect or inconsistent records.
Group lifecycle	Users can create and join groups, but leaving a group or removing members who still have outstanding debts is not supported in the first version. SplitNest also does not manage real money in a shared group fund or allow users to repay a debt in smaller installments.

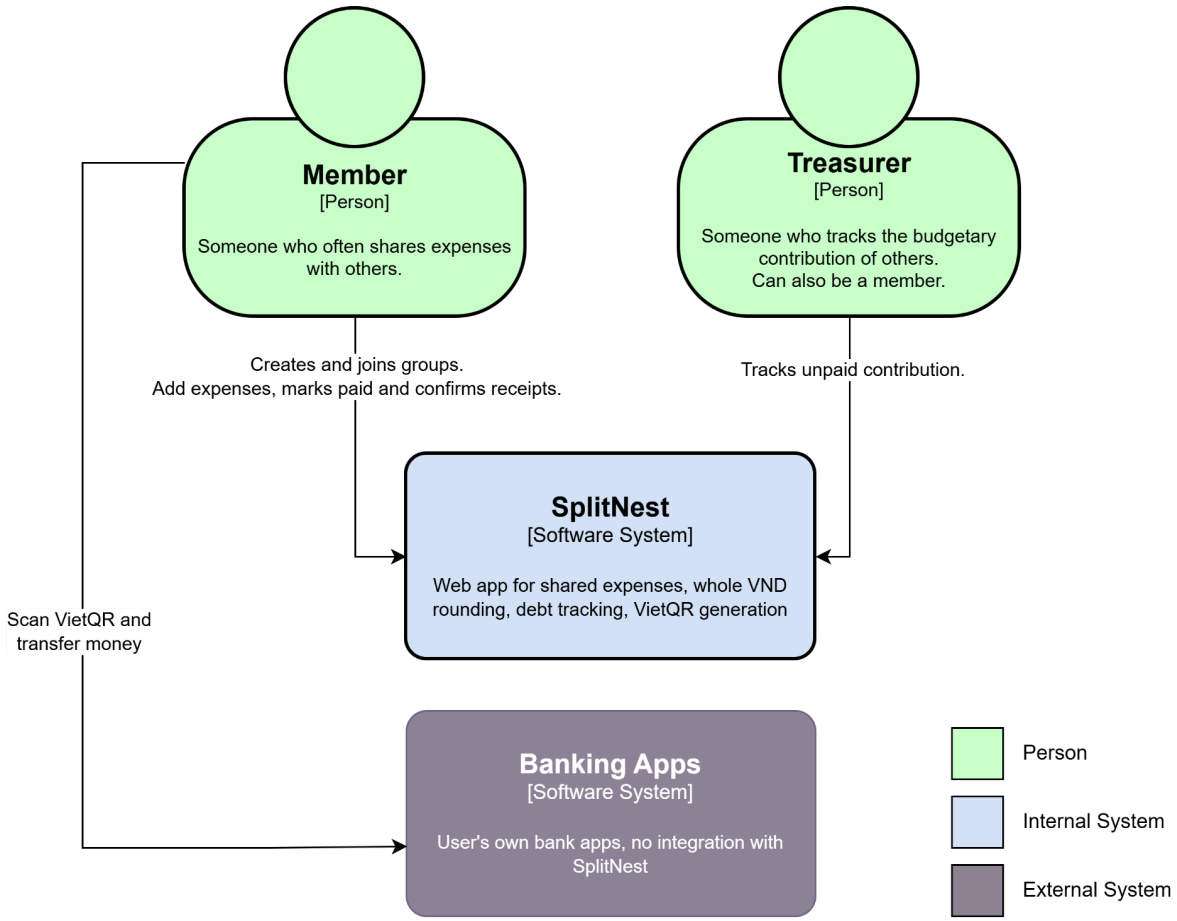
Out of scope:

+ Automatic bank transfers, bank-API connections, or automatic verification of payment receipts.

+ Support for currencies other than VND and native Android/iOS applications.
+ Email/push debt reminders (F07), in-app chat, and debt-related notes (F08).

3 Proposed Solution

3.1 Preliminary Architecture



3.2 Technology Stack

Layer	Technology	Reason for choice
Front end	React	Reusable UI components, a large community, and easy API integration.

Back end	Flask	Lightweight and easy to develop RESTful APIs quickly.
Database	MongoDB Atlas	Flexible data structure and free cloud database options.
Hosting / deployment	Vercel (frontend) Render (backend)	Easy deployment and GitHub integration.
Testing & CI	Pytest, Jest, Github Action	Automated testing for backend and frontend code.

3.3 Hardware and Operating Environment

- **Development:** laptops with Node.js, Python 3.x, Git and VS Code.
- **Testing:** Chrome, Edge and Firefox on desktop, plus a phone browser.
- **Runtime:** Vercel and Render free tiers (note Render's cold-start delay), and Atlas M0.
- **End users:** any modern browser and an internet connection.

4 Project Plan

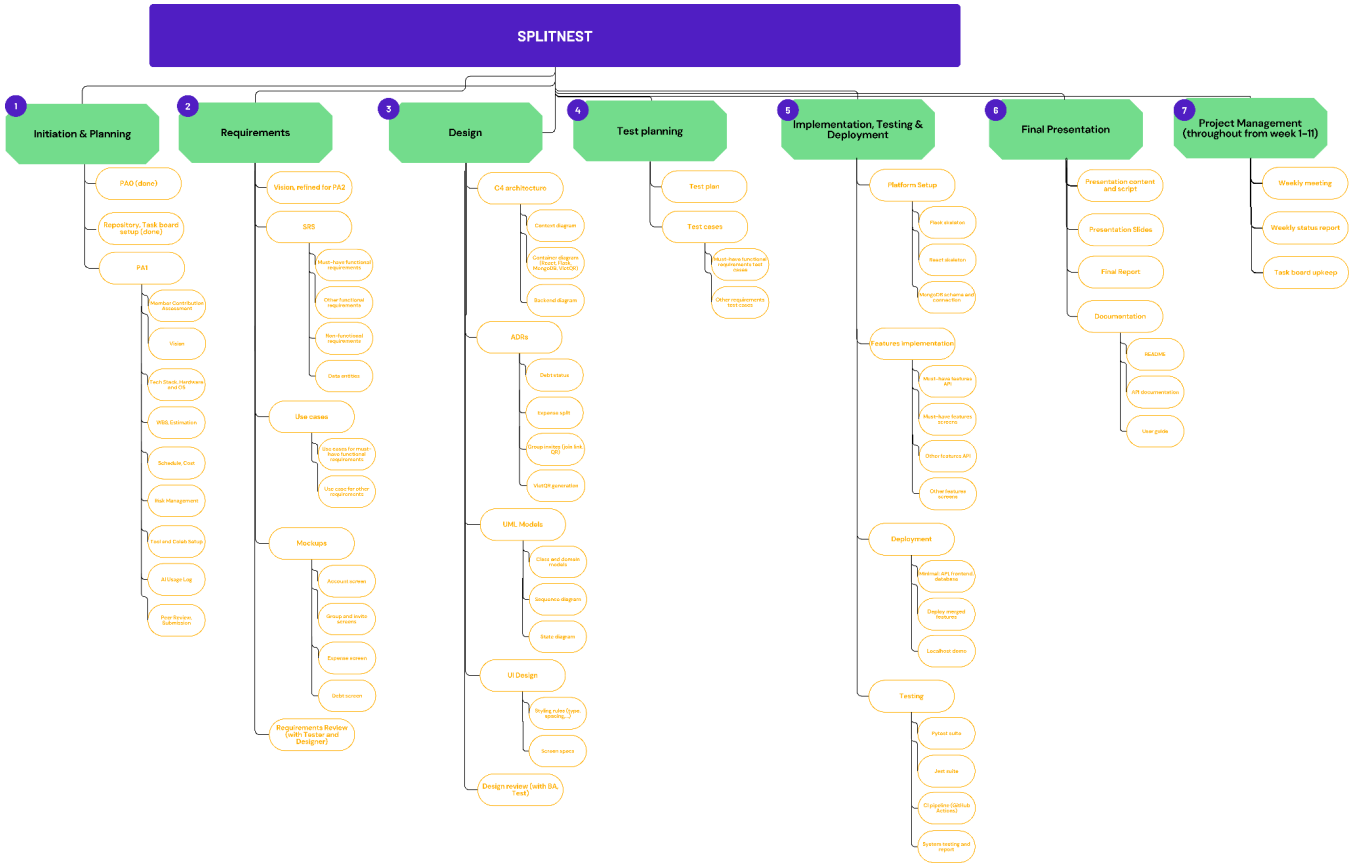
4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria (fill in)
PA1	Initiation & planning	End of week 3	Vision, position statement, stakeholders and MoSCoW-prioritised features with out-of-scope list are complete. The WBS has at least 3 levels and each work package has an owner and an effort estimate with its method. The Gantt chart shows dependencies, PA deadlines as milestones and the critical path. At least 5 risks have RE computed, a mitigation and an owner. The repository (protected main, template folder structure), Jira board with tasks, and Discord channel are working. The contribution table and AI usage log are included.
PA2	Requirements	End of week 5	Functional requirements are numbered, grouped and prioritised. Non-functional requirements are measurable, and domain rules and acceptance criteria are written. The use case model and the use case specifications (pre/post-conditions, main, alternative and exception flows) are complete. Mockups cover the main use cases. The traceability matrix is done and the review record has its issues resolved.
PA3	Design	End of week 7	Architecture is justified, with C4 context and container, component and deployment diagrams and at least 3 ADRs. The class, sequence and state diagrams are consistent with each other, and class specifications are written. The ERD, table specifications, normalisation and indexes are done. The screen flow, key screen specifications and accessibility checklist are done. The design traces to the PA2 requirements, and changes to requirements are listed.
PA4	Test planning	End of week 8	The test plan covers scope, levels, types, techniques, environment, entry/exit

			criteria, defect management and risks. EP, BVA and decision tables are applied to the right features. Test cases have title, preconditions, data, steps and expected results. Every Must-have requirement is covered by at least one test case. The deployment skeleton is live.
PA5	Implementation, testing & deployment	End of week 10	All Must-have features (F01–F04) work, are deployed and publicly accessible, and a demo video is provided. Test cases are executed and results and defects recorded, and exit criteria are evaluated. Unit tests have a coverage report, at least 2 use cases × 2 scenarios are automated, and CI runs on pull requests. The retrospective is done.
Final	Presentation & demo	Week 11	The finished project is presented, with every member speaking at least 2 minutes and the key scenario working live.

4.2 Work Breakdown Structure (WBS)



4.3 Estimation

WBS ID	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1	PA0 idea and team contract	Team	0	Done
1.2	Repository and task board setup	Lại Phan Nhật Nam	0	Done
1.3	PA1 document	-	-	Sum of 1.3.1 to 1.3.9
1.3.1	Member contribution assessment	Lại Phan Nhật Nam	1	3-point estimate. O=1, M=1, P=1 → E=1
1.3.2	Vision (section 2)	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4
1.3.3	Tech stack, hardware and OS (section 3)	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4
1.3.4	WBS and estimation (sections 4.2, 4.3)	Nguyễn Hoàng Tiến	4	3-point estimate. O=2, M=4, P=6 → E=4
1.3.5	Schedule and cost (sections 4.4, 4.6)	Lại Phan Nhật Nam	6	3-point estimate. O=4, M=6, P=8 → E=6. Depends on 1.3.4
1.3.6	Risk management (section 5)	Ngô Phúc Khang	3	3-point estimate. O=1, M=3, P=5 → E=3
1.3.7	Tools and collaboration setup (section 6)	Lại Phan Nhật Nam	2	3-point estimate. O=1, M=2, P=3 → E=2
1.3.8	AI usage log (section 7)	Trần Văn Nam	1	3-point estimate. O=1, M=1, P=1 → E=1
1.3.9	Peer review and submission	Lại Phan Nhật Nam	2	3-point estimate. O=1, M=2, P=3 → E=2.
2.1	Vision, refined for PA2	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4.
2.2	SRS	-	-	Sum of 2.2.1 to 2.2.4
2.2.1	Must-have functional requirements	Phạm Hoàng Khôi Nguyên	6	3-point estimate. O=4, M=6, P=8 → E=6.
2.2.2	Other functional requirements	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4.

2.2.3	Non-functional requirements	Phạm Hoàng Khôi Nguyên	3	3-point estimate. O=1, M=3, P=5 → E=3.
2.2.4	Data entities	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4.
2.3	Use cases	-	-	Sum of 2.3.1 and 2.3.2
2.3.1	Use cases for must-have requirements	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4.
2.3.2	Use cases for other requirements	Phạm Hoàng Khôi Nguyên	2	3-point estimate. O=1, M=2, P=3 → E=2
2.4	Mockups	-	-	Sum of 2.4.1 to 2.4.4
2.4.1	Account screen	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4
2.4.2	Group and invite screens	Trần Văn Nam	6	3-point estimate. O=4, M=6, P=8 → E=6. Includes QR join
2.4.3	Expense screen	Trần Văn Nam	6	3-point estimate. O=4, M=6, P=8 → E=6
2.4.4	Debt screen	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4
2.5	Requirements review (with Tester and Designer)	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4.
3.1	C4 architecture	-	-	Sum of 3.1.1 to 3.1.3
3.1.1	Context diagram	Trần Văn Nam	3	3-point estimate. O=2, M=3, P=4 → E=3
3.1.2	Container diagram (React, Flask, MongoDB, VietQR)	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4
3.1.3	Backend diagram	Nguyễn Hoàng Tiến	4	3-point estimate. O=2, M=4, P=6 → E=4
3.2	ADRs	-	-	Sum of 3.2.1 to 3.2.4
3.2.1	Debt status	Nguyễn Hoàng Tiến	3	3-point estimate. O=2, M=3, P=4 → E=3
3.2.2	Expense split	Nguyễn Hoàng Tiến	5	3-point estimate. O=3, M=5, P=7 → E=5.

3.2.3	Group invites (join link, QR)	Nguyễn Hoàng Tiến	3	3-point estimate. O=2, M=3, P=4 → E=3
3.2.4	VietQR generation	Nguyễn Hoàng Tiến	3	3-point estimate. O=2, M=3, P=4 → E=3
3.3	UML models	-	-	Sum of 3.3.1 to 3.3.3
3.3.1	Class and domain models	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4
3.3.2	Sequence diagram	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4
3.3.3	State diagram	Trần Văn Nam	3	3-point estimate. O=2, M=3, P=4 → E=3.
3.4	UI design	-	-	Sum of 3.4.1 and 3.4.2
3.4.1	Styling rules (type, spacing, ...)	Trần Văn Nam	3	3-point estimate. O=2, M=3, P=4 → E=3
3.4.2	Screen specs	Trần Văn Nam	3	3-point estimate. O=2, M=3, P=4 → E=3.
3.5	Design review (with BA and Tester)	Trần Văn Nam	3	3-point estimate. O=2, M=3, P=4 → E=3.
4.1	Test plan	Ngô Phúc Khang	6	3-point estimate. O=4, M=6, P=8 → E=6
4.2	Test cases	-	-	Sum of 4.2.1 and 4.2.2
4.2.1	Must-have requirements test cases	Ngô Phúc Khang	6	3-point estimate. O=4, M=6, P=8 → E=6
4.2.2	Other requirements test cases	Ngô Phúc Khang	4	3-point estimate. O=2, M=4, P=6 → E=4.
5.1	Platform setup	-	-	Sum of 5.1.1 to 5.1.3
5.1.1	Flask skeleton	Nguyễn Hoàng Tiến	4	3-point estimate. O=2, M=4, P=6 → E=4
5.1.2	React skeleton	Trần Văn Nam	4	3-point estimate. O=2, M=4, P=6 → E=4. Backup developer takes this
5.1.3	MongoDB schema and connection	Nguyễn Hoàng Tiến	6	3-point estimate. O=4, M=6, P=8 → E=6

5.2	Features implementation	–	–	Sum of 5.2.1 to 5.2.4
5.2.1	Must-have features API	Nguyễn Hoàng Tiến	32	3-point estimate. O=24, M=32, P=40 → E=32.
5.2.2	Must-have features screens	Trần Văn Nam	32	3-point estimate. O=24, M=32, P=40 → E=32.
5.2.3	Other features API	Nguyễn Hoàng Tiến	16	3-point estimate. O=12, M=16, P=20 → E=16.
5.2.4	Other features screens	Trần Văn Nam	16	3-point estimate. O=12, M=16, P=20 → E=16
5.3	Deployment	–	–	Sum of 5.3.1 to 5.3.3
5.3.1	Minimal deployment: API, frontend, database	Nguyễn Hoàng Tiến	8	3-point estimate. O=6, M=8, P=10 → E=8. Due week 8
5.3.2	Deploy merged features	Nguyễn Hoàng Tiến	6	3-point estimate. O=4, M=6, P=8 → E=6. Repeats for each merge, weeks 8 to 10
5.3.3	Localhost demo fallback	Nguyễn Hoàng Tiến	3	3-point estimate. O=2, M=3, P=4 → E=3. Contingency for risk R4
5.4	Testing	–	–	Sum of 5.4.1 to 5.4.4
5.4.1	Pytest suite	Nguyễn Hoàng Tiến	8	3-point estimate. O=6, M=8, P=10 → E=8
5.4.2	Jest suite	Ngô Phúc Khang	8	3-point estimate. O=6, M=8, P=10 → E=8
5.4.3	CI pipeline (GitHub Actions)	Ngô Phúc Khang	5	3-point estimate. O=3, M=5, P=7 → E=5
5.4.4	System testing and report	Ngô Phúc Khang	14	3-point estimate. O=10, M=14, P=18 → E=14
6.1	Presentation content and script	Lại Phan Nhật Nam	6	3-point estimate. O=4, M=6, P=8 → E=6.
6.2	Presentation slides	Phạm Hoàng Khôi Nguyên	8	3-point estimate. O=6, M=8, P=10 → E=8
6.3	Final report	Lại Phan Nhật Nam	14	3-point estimate. O=10, M=14, P=18 → E=14.

6.4	Documentation	-	-	Sum of 6.4.1 to 6.4.3
6.4.1	README	Lại Phan Nhật Nam	3	3-point estimate. O=2, M=3, P=4 → E=3
6.4.2	API documentation	Nguyễn Hoàng Tiến	4	3-point estimate. O=2, M=4, P=6 → E=4
6.4.3	User guide	Phạm Hoàng Khôi Nguyên	4	3-point estimate. O=2, M=4, P=6 → E=4
7.1	Weekly meeting	Lại Phan Nhật Nam	11	3-point estimate. per team-week O=1, M=1, P=1 → E=1 h/week × 11 weeks = 11. Team total
7.2	Weekly status report	Lại Phan Nhật Nam	18	3-point estimate. per week O=1, M=2, P=3 → E=2 h/week × 9 weeks (weeks 2 to 10) = 18
7.3	Task board upkeep	Lại Phan Nhật Nam	5.5	3-point estimate. per week O=0.5, M=0.5, P=0.5 → E=0.5 h/week × 11 weeks = 5.5

4.4 Schedule

WBS ID	Duration	Dependencies	Critical Path	Week 1	2	3	4	5	6	7	8	9	10	11
1.1	4d	-	Yes											
1.2	3d	1.1	Yes											
1.3	7d	1.1, 1.2	Yes											
2.1	2d	1.3	Yes											
2.2	7d	2.1	Yes											
2.3	3d	2.2	Yes											
2.4	11d	2.1 (parallel w/2.2)	No											
2.5	2d	2.2 → 2.4	Yes											
3.1	5d	2.5	No											
3.2	7d	2.5	Yes											
3.3	5d	2.2.4, 3.1, 3.2	Yes											
3.4	3d	2.4, 2.5	No											
3.5	1d	3.1 → 3.4	Yes											
4.1	5d	2.2	No											

4.2	9d	2.5, 4.1	No											
5.1	5d	3.3.1 (before 3.5)	Yes											
5.2.1 → 5.2.2	12d	3.5, 5.1	Yes											
5.3.1	3d	5.2.1, 5.2.2	Yes											
5.2.3 → 5.2.4	7d	5.2.1, 5.2.2	Yes											
5.3.2 → 5.3.3	14d	5.3.1, 5.2.3, 5.2.4	No											
5.4.1 → 5.4.3	18d	5.1, 4.2	No											
5.4.4	7d	5.2.3, 5.2.4, 5.4.1, 5.4.2	Yes											
6.4	6d	5.2.3, 5.2.4	No											
6.1 → 6.3	7d	5.4.4, 6.4	Yes											
7.1 → 7.3	11w	Recurring every week	Support											

* Blue for progress and Yellow for milestones (the PAs).

4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team lead / Project manager	Lại Phan Nhật Nam	Ngô Phúc Khang	Plan, assign tasks, weekly status report, contact point
Business analyst	Phạm Hoàng Khôi Nguyên	Nguyễn Hoàng Tiến	Elicit and document requirements, organise reviews
Designer / Architect	Trần Văn Nam	Lại Phan Nhật Nam	Architecture, class, data and UI design
Developers	Nguyễn Hoàng Tiến	Trần Văn Nam	Implement, unit test, review pull requests
Tester	Ngô Phúc Khang	Phạm Hoàng Khôi Nguyên	Test plan, test cases, system testing, test report

4.6 Human Resources and Cost Estimate

The project cost is estimated using the total effort of the work packages multiplied by an assumed blended labor rate. Based on the estimates in Section 4.3, the project requires approximately 374.5 person-hours, including project coordination and recurring reporting activities.

For planning purposes, a blended labor rate of ₫100,000 per person-hour is assumed to represent the combined effort of development, requirements analysis, UI/UX design, testing, and project coordination. The estimated labor cost is therefore:

Labor cost = 374.5 person-hours × ₫100,000/person-hour = ₫37,450,000.

The project plans to use the free tiers of Vercel, Render, MongoDB Atlas, GitHub, and Jira. Therefore, the estimated cost of these tools and hosting services is ₫0, provided that the project remains within their free-tier limits.

A 10% contingency reserve of ₫3,745,000 is included to account for unexpected integration, testing, or deployment work. The resulting estimated project budget is ₫41,195,000.

These figures are preliminary planning estimates. The actual cost may vary depending on the team's final effort, the number of integration cycles, and whether the selected hosting services incur charges.

5 Risk Management

ID	Risk	Type	Probability	Loss	RE	Mitigation / contingency	Owner
R1	Key member unavailable during PA5	People	2	4	8	For each role, there is a primary assignee and a backup person; all documentations and tasks progress must be updated so the backup person	Lại Phan Nhật Nam

						can take over immediately whenever necessary	
R2	Underestimation of coding and integration effort	Estimation	4	4	16	Making sure completing the for "must-have" features, any "should-have" or "could-have" features will be excluded from the scope if time is insufficient	Nguyễn Hoàng Tiến
R3	Requirements mismatch / scope creep between phases	Requirements	3	4	12	Tester and Degisner join in the review of SRS documentation with BA before submitting the submission	Phạm Hoàng Khôi Nguyên
R4	Deployment & Free-tier hosting environment issues	Technology	3	4	12	Deploy a minimal version to a live environment, also prepare a contingency plan to conduct a demo via localhost on a team member's device in case of network issues with the hosting server	Nguyễn Hoàng Tiến
R5	Uneven workload and delayed internal submissions	Organisational	2	3	6	Strictly adhere to the Team Contract: Complete tasks 12 hours before the submission deadline. If a task is late to complete, the matter will be addressed in a team meeting and documented in the Weekly Status Report, resulting in a deduction from the individual's contribution score	Lại Phan Nhật Nam

6 Tools & Collaboration Setup

One of the main goals of the project is to practise teamwork for professional software development. The use of tools is monitored and graded (teamwork component). The team must use:

- ✔ Moodle – announcements and PA submissions.
- ✔ Communication – Slack, MS Teams or Discord: one channel per team; the TA is a member.
- ✔ Task board – GitHub Projects (recommended) or Jira: every WBS work package is a task with an owner and due date.
- ✔ Git repository – GitHub (recommended), GitLab or Bitbucket; the instructor and TA have access:
  - main branch is protected; changes are merged through pull requests with at least one review
  - README (description, setup), .gitignore; never commit secrets (.env, API keys, passwords)
  - commit messages follow the Conventional Commits style (feat:, fix:, docs:, test:, ...)
- ✔ Weekly Status Meeting (WBS 7.1): The team holds a mandatory 15-minute synchronous stand-up meeting every Wednesday at 21:30 (GMT+7) via Google Meet / Discord to review completed tasks, update the task board (WBS 7.3), and flag blockers.
- ✔ Weekly Status Report Deadline (WBS 7.2): Following the meeting, the Project Manager (Lại Phan Nhật Nam) compiles the weekly status report using the course template and

commits it to /docs/management/weekly/ by every Wednesday at 23:59 (GMT+7), running from Week 3 through Week 10.

Repository structure:

Folder	Content
/src	Source code
/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 … /pa/PA5	One sub-folder per PA submission (PDF + source files)

7 AI Usage Log

#	Member	Tool	Task / purpose	How the output was verified and changed
1	24127459	ChatGPT	Understand my assigned tasks for the project.	Checked the response against the assigned tasks.
2	24127459	ChatGPT	Check if the proposed technologies and hosting options for SplitNest are suitable.	Reviewed the suggestions and noted what needed further checking.
3	24127459	Gemini	Check if the frontend, backend, database, hosting, and testing tools work well together.	Reviewed the results and noted possible compatibility issues.
4	24127459	ChatGPT	Find official documentation to check if the proposed technologies are compatible.	Compared the suggestions with official documentation and noted what still needed checking.
5	24127308	ChatGPT	Review the PA1 materials and break down the remaining work into smaller tasks.	Checked the tasks against the PA1 requirements and clarified anything that was unclear.
6	24127308	ChatGPT	Review the WBS draft and understand the difference between Level 2 deliverables and Level 3 work packages.	Used the feedback to make the WBS structure clearer and easier to follow.
7	24127308	ChatGPT	Fill in the estimation table for all tasks in the WBS.	Compared the table with the WBS to make sure no tasks were missed.
9	24127295	ChatGPT	Review whether the first 2.1 problem statement was suitable for a proposal.	The first draft felt too long and was hard to skim. I asked for a clearer flow from the problem to the solution, value, and constraints, while keeping the lecturer's 1–2 page guidance in mind.
10	24127295	ChatGPT	Compare the suggested features with our submitted PA0 Project Idea.	I checked PA0 again to avoid changing our original idea without discussion. I kept the four Must-have flows and treated VietQR, history, reminders, and chat as optional or deferred.
11	24127295	ChatGPT	Question the proposed scope against our team's resources and timeline.	I considered what five students could reasonably build in weeks 8–10. I kept bank details as text and the pay-then-confirm process instead of assuming we could integrate with real banks.

12	24127295	ChatGPT	Review stakeholders, user roles, and feature priorities from a BA perspective.	I checked that organizer, payer, and receiver are roles in a group, not necessarily separate account types. I also distinguished an invitation QR code from VietQR. The proposed priorities still need team review.
13	24127455	Claude	Drafted a C4 context diagram and suggested treasurer/member roles.	Rejected the generated diagram and the "treasurer creates groups" role. Redraw the diagram in draw.io, with the treasurer as a member who can also track unpaid contributions, matching the PA0 target-user table.
14	24127455	Claude	Reviewed the WBS and estimation table.	Recomputed the effort total and per-person load. Fix typos found.
15	24127455	Claude	Drafted phase-gate exit criteria and the hardware/environment bullets	Checked the criteria against the rubric's PA1–PA5 rows and the course schedule.
16	24127409	Gemini	Brainstorm project risks and mitigations for Section 5 (Risk Management).	Reviewed the suggested risks against the 5 rubric categories. Adjusted the Probability and Loss values to fit the project scale, mapped mitigation actions to the Team Contract, and assigned specific owners.